

Scheduling and Waiting
M2 SETI-B4 Embedded Linux

Maria Mushtaq
Maria.mushtaq@telecom-paris.fr
2022-2023

Institut Mines-Télécom

Plan

- Introduction
- Putting a task to sleep
- Wake up call
- Variant: exclusive waiting
- Conclusion

Tasks

- A task is the basic element used by the scheduler
- It is represented by the struct task_struct
- A task can be in one of the following states

  - TASK_RUNNING : the task is either currently running or ready to be executed
  - Sleeping : it is not considered by the scheduler
  - TASK_INTERRUPTIBLE : waiting interruptable by the expected event or any signal
  - TASK_KILLABLE : wait interruptible by the expected event or a SIGKILL signal
  - TASK_UNINTERRUPTIBLE : wait interruptible only by the expected event
  - TASK_DEAD : zombie

Tasks

- When a task is interrupted (interrupt from hardware, from an exception or from a system call), the processor switches to kernel space and hands over to the kernel
- This kernel code executes in the context of the task that has just been interrupted (the virtual address space is that of the task, current points to the structure representing this task, etc.)
- At the end of processing, the original task resumes execution, unless it has been put to sleep, stopped, or a higher priority task is scheduled in its place

Plan

- Introduction
- Putting a task to sleep
- Wake up call
- Variant: exclusive waiting
- Conclusion

Creation of a waiting list

- To put a process on hold, you need a wait_queue_head_t queue (defined in include/linux/wait.h)
- This structure can be initialized

  - Statically

```
wait_queue_head_t queue;

DECLARE_WAIT_QUEUE_HEAD(queue);
```

  - Dynamically

```
wait_queue_head_t *queue;

 // Memory allocation... init_waitqueue_head(queue);
```

Putting to sleep

void int wait_event(wait_queue_head_t wait_queue , bool condition)
wait_event_interruptible(wait_queue_head_t wait_queue , bool condition)

- These two functions allow to put the current task in sleep mode on the wait_queue list
- If condition is evaluated to true at the entry of these two functions, they return immediately (the task is not put to sleep)
- If not, the task is put to sleep
  - In the TASK_UNINTERRUPTIBLE state for the first one
  - In the TASK_INTERRUPTIBLE state for the second one

Putting to sleep

```
void int wait_event(wait_queue_head_t wait_queue , bool condition)
int wait_event_interruptible(wait_queue_head_t wait_queue , bool condition)
```

- Once put to sleep, these functions will return only if
  - The queue is explicitly woken up (see later in the course) and, when the task becomes active again, condition is set to true
  - Otherwise, the task is immediately put back to sleep (and the function does not return immediately)
  - Or, in the case of the second function, a signal is received by the task
  - In this case, the function returns immediately (even if condition is false or if the list has not been woken up) and returns -ERESTARTSYS

Putting to sleep

void int wait_event(wait_queue_head_t wait_queue , bool condition)
int wait_event_interruptible(wait_queue_head_t wait_queue , bool condition)

- The use of the interruptible variant is strongly recommended
- Most of the time, if you are in a callback function following a system call (for example the functions foo_read, foo_write...), and wait_event_interruptible returns -ERESTARTSYS, you just have to return -ERESTARTSYS by your function
  - This will either result in an automatic callback of your function, or in a failure of the system call with the error -EINTR seen by the application

Putting to sleep (variants)

int wait_event_timeout(wait_queue_head_t wait_queue , bool condition , timeout)
int wait_event_interruptible_timeout(wait_queue_head_t wq, bool condition , timeout)

- These two variants add an early exit condition in case of expiration of the timeout expressed in jiffies
- They return

  - 0 : timeout expired and false condition

  - 1 : true condition at the end of the timeout or at the wake-up of the waiting list

  -ESTARTSYS : signal received (for wait_event_interruptible_timeout)

Plan

- Introduction
- Putting a task to sleep
- Wake up call
- Variant: exclusive waiting
- Conclusion

Wake up call

```
void wake_up(wait_queue_head_t *q);
```

- All the pending tasks on this list are woken up
- When the scheduler gives them time to run, they will each evaluate their waiting condition (condition passed to the wait_event... functions)
    - For the tasks whose condition is verified, the wait is over, the call to the wait_event... function ends
    - For the others, they are put back to sleep
- The call to the wake up function is typically made
- in an interrupt handler to wake up the task(s) that were waiting for this event

Wake up call

```
void wake_up_interruptible(wait_queue_head_t *q);
```

- This function only wakes up tasks that have been wait_event_interruptible*.
- By convention, if you use wait_event_interruptible*, use wake_up_interruptible

  (you can also use wake_up)

Plan

- Introduction
- Putting a task to sleep
- Wake up call
- Variant: exclusive waiting
- Conclusion

Exclusive waiting

- In some situations, several tasks may be waiting for an event but only one of these tasks will be able to process this event
- It is therefore unnecessary to wake them all up

Stand-by mode

- The functions seen before put the task in non-exclusive wait
- int

  wait_event_interruptible_exclusive(wait_queue_head wait_queue, bool

  condition) places the task in exclusive wait

Wake up call

- wake_up(wait_queue_head_t *q) and wake_up_interruptible(wait_queue_head_t *q) wake up all non-exclusive waiting tasks and a single exclusive waiting task (if any)
- wake_up_nr(wait_queue_head_t *q, int nr) and wake_up_interruptible_nr(wait_queue_head_t *q, int nr) wake up all non-exclusive pending tasks and at most nr exclusive pending tasks
- wake_up_all(wait_queue_head_t *q) and wake_up_interruptible_all(wait_queue_head_t *q) wake up all non-exclusive waiting tasks and all exclusive waiting tasks

Plan

- Introduction
- Putting a task to sleep
- Wake up call
- Variant: exclusive waiting
- Conclusion

Example od use in a driver

- A waiting list is declared in the foo_device structure to store processes waiting for data from the device
- In the foo_read callback function, the driver looks to see if data is currently available. If not, the process is put on hold
- In the interrupt handler called when data is available in the device, the driver wakes up the waiting processes